

Tema 6: Sortering

Wilhelm Durelius widu7139

17 februari 2026

1 Enkellänkade listor

Vad ska man tänka på om man ska välja en sorteringsalgoritm som ska användas för data som kommer i formen av en enkellänkad lista?

1.1 Svar

Om datamängden är liten och man står inför valet av någon av de enklare $O(n^2)$ algoritmerna, så bör man tänka på att det är meckigt och ofta inte värt det att använda insertion sort just för enkellänkade listor, då algoritmen går **bakåt** i listan.

Om datamängden är stor och man står inför valet av någon av de mer komplexa $O(n \log n)$ algoritmerna, så bör man tänka på att om datan är hyfsat sorterad i början av listan, eller om listan är omvänt sorterad, blir det väldigt ineffektivt att använda Quick Sort då man får ett dåligt pivot-värde från det första elementet i den länkade listan.

2 Insertion kontra Selection sort

Vad är skillnaderna mellan insertion och selection sort, och när ska man använda den ena eller den andra?

2.1 Svar

Både insertion och selection sort använder koncepten av en osorterad och sorterad del av en lista, skillnaden är att insertion placerar osorterade element på rätt plats i den sorterade delen, medan selection sort konstant placerar det minsta elementet i den osorterade delen längst bak i den sorterade. För att hitta det minsta elementet måste också selection sort traversera hela listan, vilket insertion sort inte behöver göra, utan den tar element för element.

Insertion sort är därmed bra på data som är hyfsat sorterad, men gör många förflyttningar. Best case för insertion sort är $O(n)$ på en redan sorterad lista.

Selection sort är bra när man vill göra få förflyttningar, då varje element flyttas enbart en gång. Då har vi n förflyttningar, men n^2 jämförelser.

Båda dessa ökar dock i tidskomplexitet snabbt vid större datamängder, med tanke på att de är $O(n^2)$ algoritmer.

3 Bucket sort

Vad är en bucket sort, hur fungerar den och vad är användningssområdet?

3.1 Svar

Bucket sort används när du ska sortera någonting baserat på en egenskap som är mycket färre än antalet element, och har överlapp.

Ett exempel på detta är sortering av födelsemånad inom en stor grupp människor, till exempel inom en kommun. Tusentals människor, men enbart 12 möjliga värden. Då är bucket sort perfekt. Vi delar då in alla människorna i **hinkar** baserat enbart på vilken månad denne är född.

En väldigt förenklad visualisering av en implementation av bucket sort kan då vara att man har en Map, där *key* är månaden, och *value* är en lista med människor som är födda den månaden. Varje gång vi vill lägga till en människa i en bucket, gör vi då

```
bucket[person.year].append(person)
```

för att lägga till en person på rätt ställe.

Värt att tänka på är att om antalet element är väldigt få, är bucket sort nästan aldrig värt det, då det blir för stor overhead och överlappet blir litet.

Vid en bra implementering av bucket sort blir tidskomplexiteten $O(n)$.

4 Varför $O(n \log n)$?

Varför är de generella sorteringsalgoritmerna aldrig snabbare än $O(n \log n)$?

4.1 Svar

En lista med n element har fakulteten av n (alltså $n!$) sätt att ordnas på. En av dessa ordningar är den korrekt sorterade versionen. Varje jämförelse har möjligheten att reducera kvarvarande alternativ med hälften, antalet halveringar man behöver för att komma ner till 1 kvarvarande alternativ är ungefär $\log(n!)$, vilket likställs med $O(n \log n)$.